



模板使用

▼ 模板推导指引

模板推导指引（Class Template Argument Deduction Guide，简称 CTAD）是 C++17 引入的语法，允许编译器根据构造函数的参数自动推导模板参数，而不需要显式写出模板参数列表。

在 C++17 中，一些简单的模板类可以被编译器根据构造函数的参数类型自动推导出模板类需要实例化的类型，但在有些复杂的模板中，编译器无法从构造函数参数直接推导出正确的模板参数，这时就需要推导指引（Deduction Guide）。

C++

```
template<typename... Args>
ClassName(Args...) -> ClassName<DeducedArgs...>;
```

- 左边是构造函数参数列表；
- `->` 右边是模板参数推导结果；
- 这条语句不是构造函数，是推导规则声明。

▼ 示例

C++

```
template<class... Ts>
struct Overloaded : Ts... {
    using Ts::operator()...;
};

template<class... Ts>
Overloaded(Ts...) -> Overloaded<Ts...>; // ✅ 这就是推导指引
```

当编译器使用 `Overloaded(Ts...)` 类型的构造函数时，会推测当前需要实例化的模板类型为 `Overloaded<Ts...>`，所以在初始化时不再需要显式指定 `Overloaded<Ts...>`，只需要使用 `Overloaded` 调用构造函数进行初始化即可。

▼ 类模板特化

在主模板之后，通过显式指定模板参数方式再次定义模板的方式，叫做特化。

此时若 `template` 中还存在模板参数，则是偏特化，不存在则是全特化。

```
C++

// 主模板
template <class T1, class T2>
class A{
    T1 data1;
    T2 data2;
};

// 全特化类模板，显式指定<int, double>，且template 中不存在模板参数
template <>
class A<int, double>{
    int data1;
    double data2;
};

// 偏特化类模板，显式指定<int, T>，但是template 中还有模板参数T
template <class T>
class A<int, T>{
    int data1;
    T data2;
};
```

全特化：所有的模板参数都被指明了具体的类型，不再有 `typename...` 或 `Args` 这种待定变量。

偏特化：限制了参数的范围，但其中仍然包含未确定的模板参数，比如 `Args`。

如果全特化和偏特化存在重合的部分，会优先使用特化更具体的全特化模板。

指针特化与引用特化也属于偏特化，因为其满足了显式指定和仍有模板参数两个特征。

```
C++

// 左值引用特化
template <class T1, class T2>
class A<T1&, T2&>{
    T1& data1;
    T2& data2;
};

// 指针特化
template <class T1, class T2>
class A<T1*, T2*>{
    T1* data1;
    T2* data2;
};
```

对于某些容器，其本身为高阶模板，不能直接匹配 `typename T`，但是可以通过一层偏特化进行处理：

C++

```
template <typename T>
class A;

template <typename T>
class A<<std::vector<T>>>{
    // do something
};
```

▼ 函数模板特化

因为函数的重载机制，函数模板并不存在偏特化，也并不推荐使用全特化。

因为模板也可以重载，所以使用偏特化的方式来“特化”函数模板，会被视为模板重载。

如果存在模板重载，且两个重载的模板存在交集，那么全特化的模板会被视为上方最近的模板的全特化。

所以为了避免不必要的错误，不推荐使用全特化，而是使用重载来替代全特化。

全特化去掉关键字 `template<>` 就不是全特化而是重载。

但是有一种情况，即只有返回值使用模板参数时。

编译器不支持重载仅有返回值不同的函数，所以处理这种类型函数还是要使用模板的全特化。

▼ 二阶模板

二阶模板是模板的模板，即在模板类的模板参数中使用模板类。

与一阶模板使用 `typename` 定义模板类型不同，二阶模板使用 `template <typename> class` 定义二阶模板类型，`template <typename> class` 表示拥有一个模板参数的模板类，拥有两个模板参数的模板类应该使用 `template <typename, typename> class`，以此类推。

需要注意的是，确定了模板参数类型的模板类，本质是普通类，比如 `std::vector<int>` 会匹配 `typename T`，`std::vector` 才是模板类，会匹配 `template <typename, typename> class`。

比如 `template <template <typename> class Container, typename T>` 表示该模板的第一个模板参数为一个“只有一个模板参数”的模板类。

`template <typename> class` 中的 `<typename>` 只起到告诉编译器二阶模板参数表示的一阶模板类有多少模板参数，而不起到实际作用。如果想在二阶模板中对一阶模板类进行泛型定义，则需要一个其他模板参数，否则只能指定一阶模板类的类型。

```
// class.hpp
template <typename T>
class MyContainer {
private:
    T element;
public:
    MyContainer(T arg) : element(arg) {}
    T getElement() { return element; }
};
```

C++

```

template <template <typename> class Container, typename T>
class Wrapper {
private:
    Container<T> container;
public:
    Wrapper(T arg) : container(arg) {}
    T getElement() { return container.getElement(); }
};

// main.cpp
#include "class.hpp"
#include <iostream>
int main() {
    Wrapper<MyContainer, int> myWrapper(42);
    std::cout << myWrapper.getElement() << std::endl; // 输出 42
    return 0;
}

```

在该例中，`MyContainer` 拥有一个模板参数，想要在 `Wrapper` 中对其进行泛型定义，需要为 `Wrapper` 添加一个模板参数，否则就无法像例中一样使用 `Container<T>`，而只能像 `Container<int>` 一样确定类型。

▼ 类型萃取

类型萃取主要发生在编译器，用于提供编译期查询、提取或修改类型信息的机制。

在泛型编程中，想要为不同的类型匹配不同的逻辑，一般需要使用类型萃取。

根据 C++ 标准的演进，实现类型萃取主要有四种手段：

- 模板偏特化：底层技术，通过对特定类型（如指针 `T*`）编写特化版本来“提取”内部信息。
- `std::enable_if` (C++11 之前)：SFINAE(Substitution Failure Is Not An Error)思想
- `<type_traits>` (C++11/14)：标准库提供的元函数集合（如 `std::is_integral<T>`），通过模板特化实现。
- `if constexpr` (C++17)：编译期条件分支。
- `concepts-requires` (C++20)：语义级的类型约束，通过限定模板参数特征，直接在函数签名层面实现类型过滤。

一个标准的萃取类（Trait Class）或协议类，通常包含以下三类成员：

成员名称	类型/格式	核心用途
<code>value</code>	<code>static constexpr bool</code>	属性判断。标识类型是否符合某种特征（如 <code>is_pointer::value</code> ）。
<code>value_type</code>	<code>using / typedef</code>	类型提取。记录或暴露底层原始类型，常用于迭代器指向的元素类型。
<code>xxxx_tag</code>	<code>struct</code> （空结构体）	标签分发 (Tag Dispatching)。配合重载机制，在多个算法实现间进行静态路由。

标签分发是类型萃取最经典的用法，通过萃取出的 `tag` 决定调用哪个重载函数：

```
C++

// 1. 定义标签
struct fast_path_tag {};
struct slow_path_tag {};

// 2. 萃取机 (Traits)
template <typename T>
struct my_traits {
    using route_tag = slow_path_tag; // 默认走慢路径
};

// 3. 算法实现 (Overloads)
template <typename T>
void do_work_impl(T val, fast_path_tag) { /* 硬件加速逻辑 */ }

template <typename T>
void do_work_impl(T val, slow_path_tag) { /* 通用兼容逻辑 */ }

// 4. 统一入口
template <typename T>
void do_work(T val) {
    do_work_impl(val, typename my_traits<T>::route_tag{});
}
```

以迭代器为例，STL 不直接访问迭代器的内部成员，而是通过一个中间模板层

`std::iterator_traits<Iter>` 来间接获取信息：

```
C++

template <typename Iter>
struct iterator_traits {
    using value_type = typename Iter::value_type;
    using difference_type = typename Iter::difference_type;
    using iterator_category = typename Iter::iterator_category;
    // ... 其他定义
};
```

其中的 `iterator_category` 就是决定了标签分发的空结构体类型。

```
C++

template <typename Iter>
void my_advance(Iter& it, int n) {
    using category = typename std::iterator_traits<Iter>::iterator_category;
    _my_advance_helper(it, n, category{}); // 编译器根据标签类型自动匹配重载
}

// 针对随机访问迭代器的优化版
void _my_advance_helper(Iter& it, int n, std::random_access_iterator_tag) {
    it += n;
}

// 针对普通迭代器的通用版
```

```
void _my_advance_helper(Iter& it, int n, std::input_iterator_tag) {  
    while (n--> ++it;  
}
```

在调用函数时，`category{}` 创建一个该结构体类型的一个临时右值作为函数参数，就可以匹配至对应的重载函数中。

▼ 参考笔记

[深入理解特化与偏特化 · C++ Template Tutorial \(gitbooks.io\)](#)